

Course Description

Course Name	Python Programming
Course Code	IT1104
Relevant Job Roles & Entrepreneurship Opportunities (If any)	Junior Python Developer
Relevant QP	SSC/Q0510 Software Programmer-Python (NSQF Level 4, Version 1.0, on the Scale of 10)
Relevant SDG (If any)	SDG 9 – Industry, Innovation & Infrastructure (Software & Digital Innovation)
Relevant Standards (If any)	PEP 8, Style Guide for Python Code
Competencies	1. Interpret program specifications and formulate Python-based solutions by identifying inputs, outputs, processing requirements, and suitable programming logic for simple applications. 2. Develop modular Python applications using appropriate programming constructs, functions, and built-in data structures to solve defined computational problems. 3. Apply Python coding standards and reusable programming practices through meaningful naming, modularity, portability, and readable program structure. 4. Test, debug, and refine Python programs by creating suitable test cases, identifying errors, and validating program outputs against stated requirements.

Course Outcomes	After completion of the course, the student will be able to: CO1. Analyse simple problem statements and program specifications to formulate algorithms and Python-based solutions. CO2. Develop Python programs using variables, expressions, input-output operations, decision-making constructs and iteration to solve simple programming problems. CO3. Construct Python programs using data structures like lists, tuples, sets and dictionaries to manage and process data. CO4. Develop modular Python programs by applying reusable functions and portable program logic. CO5. Test, debug, and execute Python programs using appropriate test cases to identify and correct programming errors.																									
Related World Skill-Competition	Prerequisite for Web Technologies																									
Course Structure	<table><tr><th>Total Contact Hrs. (LTP)</th><th>Days</th><th>Total L&T Hrs.</th><th>Total P Hrs.</th><th>Total SL Hrs.</th><th>Credits</th></tr><tr><td>45</td><td>10-12</td><td>15</td><td>30</td><td>45</td><td>2</td></tr></table>						Total Contact Hrs. (LTP)	Days	Total L&T Hrs.	Total P Hrs.	Total SL Hrs.	Credits	45	10-12	15	30	45	2								
Total Contact Hrs. (LTP)	Days	Total L&T Hrs.	Total P Hrs.	Total SL Hrs.	Credits																					
45	10-12	15	30	45	2																					
Assessment Scheme																										
<table><tr><th>Component</th><th>Sub-Component</th><th>Total Assessments</th><th>Marks (Max. 100)</th></tr><tr><td rowspan="5">Continuous Assessment (CA)</td><td>Assignment</td><td>2</td><td>20</td></tr><tr><td>Practical Exam</td><td>1</td><td>30</td></tr><tr><td>Report Writing</td><td>1</td><td>10</td></tr><tr><td>Vivo Voce</td><td>1</td><td>10</td></tr><tr><td></td><td></td><td></td></tr></table>							Component	Sub-Component	Total Assessments	Marks (Max. 100)	Continuous Assessment (CA)	Assignment	2	20	Practical Exam	1	30	Report Writing	1	10	Vivo Voce	1	10			
Component	Sub-Component	Total Assessments	Marks (Max. 100)																							
Continuous Assessment (CA)	Assignment	2	20																							
	Practical Exam	1	30																							
	Report Writing	1	10																							
	Vivo Voce	1	10																							

Sub Total		4	70
Terminal Assessment (TA)	Written Exam	1	30
Sub Total		1	30
Grand Total		5	100

Reference Books	1. Thareja, R. (2017). Python Programming: Using Problem Solving Approach. Oxford university press. 2. Liang, Y. D. (2022). Introduction to programming using Python. USA. 3. Deitel, P. (2019). Python for Programmers. 4. Kaswan, K. S., Dhatteval, J. S., & Balamurugan, B. (2023). Python for beginners. Chapman and Hall/CRC. 5. Shaw, Z. A. (2017). Learn python 3 the hard way: A very simple introduction to the terrifyingly beautiful world of computers and code. Addison-Wesley Professional.
------------------------	---

Detailed Syllabus

Unit	Name	Content	Hrs.
I	Introduction and Algorithms	K1 – Introduction to Programming and Algorithms K2 – Algorithm Design Methodologies K3 – Introduction to Structograms, Structograms Design Methodologies IKS1- Aryabhata's recursive sine table algorithm (~5th century CE) — Aryabhataiya	4
		A1 – Algorithm Writing for General Problems A2 – Designing structograms for given problems A3 – Dry run and validation of algorithms	7

II	Python Fundamentals	K1 – Introduction to Python, Installing Python IDEs – Python IDLE and VS Code K2 – Syntax, comments, variables, data-types, numbers, strings K3 – Operators in Python, Taking User Input	4
		A1 – Writing basic Python programs using variables and operators A2 – Practice of input-output programs in VS Code A3 – Debugging syntax and runtime errors	7
III	Control Flow and Loops	K1 – Conditional Statements – If, else, elif and Nested If else K2 – Loops in Python – For Loop, While Loop & Nested Loops	3
		A1 – Developing decision-based programs A2 – Loop-based problem solving (factorial, patterns etc.)	7
IV	Collections, Functions and Modules	K1 – Python Collections (Arrays) – List, Tuple, Sets and Dictionary; String Manipulation – Basic Operations, Slicing & Functions and Methods K2 – User Defined Functions – Defining, Calling K3 – Types of Functions, Arguments, Lambda Function; Importing Modules – Math Module	4
		A1 – Programs using lists/dictionaries and string manipulation A2 – Developing modular programs using functions A3 – Practice of lambda-based solutions; Using math module in scientific and real-world calculations	9
		Total	45

Examples of Practical Exercises

Ex 1: Develop a complete Python program for a Loan Interest Management System that accepts user inputs and ensures:

- Correct mathematical computation
- Proper input validation

- Clean formatted output
- Error-free execution

Ex 2: Develop a Python-based Student Result Processing System ensuring:

- Accurate percentage calculation
- Correct grade classification
- Proper use of lists
- Well-formatted report output

Examples of Problem-Solving Tasks

T1: Program Output Incorrect

Situation:

The student result program runs but the percentage is wrong.

Student will:

1. Diagnose root cause by checking formula and data type.
2. Identify whether issue is due to:
 - o wrong formula
 - o integer division
 - o incorrect list usage
3. Modify logic correctly.
4. Justify the correction.

T2: Loop Running Infinitely

Situation:

The menu-driven program is stuck in an infinite loop.

Student will:

1. Identify loop termination issue.
2. Decide whether to:
 - o modify loop condition OR
 - o add break statement
3. Justify the decision.
4. Implement a corrected version.

Examples of Problem-Solving Questions

Q1. Program Crash Analysis

A student executed the following Python code:

```
n = int(input("Enter number: "))  
print("Square root:", n**0.5)
```

The program crashes when the user enters 12.5.

Students will:

1. Identify the programming mistake.
2. Explain why the crash occurred.
3. Modify program for safe execution.
4. Suggest additional input validation.

Q2. Code Works in One System but Fails in Another

Situation:

The Python program runs in VS Code but gives errors in another environment.

Students will:

1. Identify possible environmental causes.
2. Explain interpreter/version dependency.
3. Suggest verification steps.
4. Evaluate risks of untested code deployment.